

FLIPSTRESS: Noise Injection Defenses Against CPU-cache-based Web Attacks

Tapan Basak¹, Hai Phan¹, Yossi Oren², and Reza Curtmola¹

¹ New Jersey Institute of Technology, Newark NJ 07102, USA

² Ben Gurion University, Israel

{tb358, reza.curtmola, phan}@njit.edu, yos@bgu.ac.il

Abstract. Side-channel attacks via the CPU-cache can leak sensitive information about web browsing users. Two prominent forms of such attacks are *targeted deanonymization*, which reveals a user’s identity, and *website fingerprinting*, which exposes which websites a user visits. In this paper, we present **FLIPSTRESS**, a defense that mitigates these attacks by injecting artificial noise to obfuscate the cache patterns and render the side-channel information ineffective. FLIPSTRESS is designed to withstand strong attacks that leverage machine learning models to interpret the cache readings. Towards this goal, we start by developing several *stressor* programs that create artificial noise by performing continuous read or write operations on cache-sized data structures. We then consider progressively stronger attackers who incorporate increasingly more information about the defense mechanism into their machine learning pipeline. Conversely, we enhance gradually the defense strategy and converge on FLIPSTRESS, which injects artificial noise by switching a randomly picked stressor at regular time intervals. We implement FLIPSTRESS in JavaScript as a browser extension meant for selective activation when users visit high-risk or sensitive websites where protection against cache-based attacks is critical. FLIPSTRESS reduces targeted deanonymization accuracy to 57.5% (base rate 50%) and website fingerprinting accuracy to 6.3% (base rate 1%), with tunable performance overhead between 31%-226%.

Keywords: side-channel attacks · noise-based defenses · web privacy.

1 Introduction

Side-channel attacks are a class of attacks that exploits indirect information leaked through the physical behavior of hardware to infer sensitive data. Rather than directly attacking cryptographic protocols or network communications, these attacks gather unintended signals — such as timing information, power consumption, or cache usage patterns — revealing insights into the system’s activities. *CPU-cache-based side-channel attacks*, a specific subset, take advantage of the CPU’s cache behavior to expose private information such as user activity or sensitive data. These attacks can be particularly dangerous because they exploit fundamental hardware characteristics, making them difficult to patch or fully defend against at the software level.

We focus on two prominent such attacks that pose a serious privacy threat to web browsing users: *targeted deanonymization* and *website fingerprinting*.

Targeted deanonymization (TD) attacks [28, 29, 32, 33] allow an attacker who controls a website to learn whether a specific target user is browsing the website. The attacker only needs to know the target through a public identifier, such as an email address, a LinkedIn user identifier, or an Instagram handle. The attack leverages *state-dependent URLs (SD-URLs)*, which return different responses depending on a user’s identity. An example of such SD-URLs are *leaky resources* [28, 32] such as images or videos, which are stored at resource-sharing services such as YouTube, Google Drive or Dropbox, and shared only with a specific target user. The attacker privately shares the resource with the target and then embeds this shared resource into the attack website. If a user visiting the attack website can access the embedded resource, this indicates that the current visitor is the intended target. These attacks are practical, scalable, and can be used as a stepping stone for executing more sophisticated attacks [33].

There are two main approaches to execute the attack. The first one uses cross-site leaks (XS leaks) [29], which are a family of mechanisms that exploit behaviors that bypass the same origin policy to enable the attack, such as status code leaks, page content leaks, and header leaks [14]. The second one uses browser-based side channels, such as the CPU cache side channel, which allow a (JavaScript-based) spy process to infer whether the visiting user is able to access the leaky resource.

This work focuses on the second approach, which uses side channels to exploit fundamental hardware characteristics and is thus harder to defend against [1, 2]. In particular, CPU side channel attacks work even if the resource-sharing service does not allow the embedding of its resources, or when browsers disable third-party cookies for embedded resources.

Website fingerprinting (WF) attacks enable an adversary to infer which websites a user is visiting by analyzing the side-channel data generated as different sites are loaded [12]. Prior work has shown that CPU cache usage is an effective source of side-channel data for this attack [27, 16, 25]. Unlike traditional website fingerprinting, which relies on network traffic analysis, cache-based fingerprinting focuses on how the computer’s cache behaves during the loading and rendering of web pages. This attack typically involves injecting malicious JavaScript into a webpage, which then monitors cache access times. Each website interacts with the cache in a distinctive way, creating identifiable patterns based on the resources (like images, scripts, or stylesheets) the website loads. By measuring cache occupancy or latency through these patterns, the attacker can “fingerprint” the websites being visited by the user.

Both targeted deanonymization and cache-based website fingerprinting exploit the lack of process isolation in the cache at the micro-architectural level, allowing an attacker to undermine user anonymity and expose browsing activity.

Priorly proposed defenses: Two fundamental approaches can be used to defend against side-channel attacks. The first is prevention, which aims at making attacks theoretically impossible by removing correlations with the side chan-

nel trace. In the context of targeted deanonymization attacks, Zaheri et al [33] proposed LEAKUIDATOR+, a browser extension which ensures that cross-site web requests are stripped of cookies and also eliminates a timing side-channel. However, this solution requires users to decide whether certain requests are legitimate (such as authentication requests) and mark them as such. This requirement may negatively impact user experience and privacy, as the user needs to spend additional effort and may not always be able to ascertain the legitimacy of requests.

The second defensive approach is mitigation, which tries to make attacks impractical by reducing the signal-to-noise ratio of the side-channel trace. Noise injection is a key tactic in this strategy, effectively obfuscating the measurement of the side-channel signals that attackers rely on.

Several works [22, 34, 9, 30, 10, 17, 20] utilize cache flush instructions to disrupt side-channel attacks, strategically clearing cache contents to thwart information leakage. While mitigation defenses have been thoroughly explored in the context of cryptanalysis side-channel attacks, they have not been as widely studied in web-based side-channel attacks. In the context of targeted deanonymization attacks, Zaheri et al. [33] considered a noise-based approach against CPU cache side channels: the user runs CPU stress tests outside the browser. Although they found this approach ineffective, we note that their conclusion only applies to the specific type of noise that was used and cannot be generalized.

Noise-based defenses have been more rigorously studied in the context of cache-based website fingerprinting (WF) attacks. *DefWeb* [25], for instance, leverages self-modifying code with precomputed noise templates during website rendering to mask actual cache usage. However, this approach lacks scalability and is difficult to maintain, as the noise templates are highly dependent on specific micro-architectures, operating systems, and the rendered websites themselves. *Cache Shaping* [16] obfuscates cache patterns through parallel read/write operations outside the browser involving dummy files on the disk. Whereas both DefWeb and Cache Shaping were designed to mitigate WF attacks, we show that they are ineffective against targeted deanonymization attacks.

This work takes on the challenge of developing **FLIPSTRESS**, a practical, portable, and tunable noise-based defense that is generalizable across diverse cache-based attack types. To build this defense, we investigate noise-based approaches to mitigate targeted deanonymization attacks. Our initial exploration focused on the **stress-ng** [6] stress test suite, which includes over 270 CPU stress tests. This investigation shows that different stressors in the suite vary in their effectiveness at mitigating attacks. Some stressors can significantly reduce attack accuracy, both when the attacker is unaware of the defense and when the attacker trains a machine learning model while the stressor is active. Based on this analysis, we identify three high-performing “quality” stressors from the suite and analyze their source code. Additionally, we develop four custom “quality” stressors that perform continuous read and write operations on cache-sized data structures. We implement these seven stressors in JavaScript and package them into a browser extension, enabling users to activate the defense seamlessly within the browser whenever they require protection.

Next, we consider progressively stronger attacks that incorporate more information about the defense mechanism into the machine learning pipeline used for the attack, and also extend the duration of the attack to capture more information about the target. In complement, we enhance gradually the defense strategy to mitigate the stronger attacks. Notably, we show that a strategy that adds artificial noise by switching to a randomly picked stressor at regular time intervals provides an adequate defense over time, even against the strongest adversaries.

In summary, we make the following contributions in this work:

- We develop seven “quality” CPU stress programs (*i.e.*, *stressors*) in JavaScript, which provide resilience against cache-based web attacks and form the basis of our noise-based defense. We independently develop four of these stressors. For the remaining three, we analyze the source code of **stress-ng** stressors to understand which kind of read and write operations provide effective artificial noise, and subsequently re-implement them in JavaScript.
- We extensively explore the targeted deanonymization attack and defense space. On the attack side, we consider progressively stronger adversarial strategies that feed the cache measurements collected through the side-channel into machine learning models to achieve attack accuracies close to 100%. On the defense side, we enhance gradually the defense strategy by increasing the amount of noise added, incorporating randomness into the generated noise, and periodically changing the type of stressor used to generate noise. Our key findings are synthesized into FLIPSTRESS, a defense strategy that adds artificial noise by activating a randomly-selected “quality” stressor at regular time intervals. FLIPSTRESS provides sustained protection over time, even against powerful adversaries, by reducing the attack’s success rate to 57.5%. This holds across multiple computer architectures (ARM and Intel x86), operating systems (Linux, Windows, MacOS), and browsers (Chrome, Firefox, Tor). In contrast, we show that existing noise-based defenses [25, 16, 33] are ineffective against the sophisticated adversaries we considered.
- We also demonstrate that FLIPSTRESS is effective against cache-based website fingerprinting attacks. Specifically, in a closed-world scenario, FLIPSTRESS reduces the attack’s success rate to as low as 6.3%.
- We implement FLIPSTRESS as a browser extension for Chrome, Firefox, and Tor, and make it publicly available [5] so that it can be used immediately by users needing protection. As it adds overhead, FLIPSTRESS is recommended for selective activation when users visit high-risk or sensitive websites where protection against cache-based attacks is critical. As such, users can activate or de-activate the defense on demand with just a button click without leaving the browser. In addition, the defense allows users to control the intensity of the generated noise by controlling the number of stressor instances. To our knowledge, this is the first noise-based defense available directly in the browser, offering a convenient solution for protecting user privacy.

Our work suggests that noise-based defenses can provide protection against CPU cache-based side-channel attacks. The rest of this paper is organized as

follows: Section 2 reviews related work, Section 3 provides necessary background, and Section 4 outlines the threat model. Section 5 explores the TD attack and defense space, detailing our iterative process to develop our proposed defense. Sections 6 and 7 present results for TD and WF attacks, respectively. Section 8 evaluates performance overhead, and Section 9 discusses the results and future directions. We conclude in Section 10.

2 Related Work

Lyu and Mishra [19] survey recent cache-based side-channel attacks and countermeasures. In cache-based website fingerprinting, noise injection has been recently explored as a countermeasure. Cache masking [27] introduces noise by allocating an LLC-sized buffer and repeatedly accessing each cache line in a loop to evict and mask browser activities. Cook et al. [7] proposed generating interrupts by scheduling thousands of activity bursts and network pings at random intervals. However, these approaches prove ineffective when attacker retrain their model using the same type of noise used by the defense [27, 16, 25].

DefWeb [25] takes a more advanced approach, using variational autoencoders (VAEs) to generate minimal noise templates and injecting precise noise into cache-based website fingerprints. Although effective in simulations, its reliance on precomputed templates tailored to specific microarchitectures makes it unscalable and challenging to adapt to diverse hardware or evolving website structures. DefWeb also struggles to obfuscate cache patterns of high-activity websites.

CacheShaping [16] adopts a different methodology by running multiple parallel processes to simulate browser rendering operations. It repeatedly reads data from dummy files into the LLC, evicts existing data, and writes the evicted data back to the files, masking real cache activities. While effective against website fingerprinting (WF) attacks, CacheShaping and the other website fingerprinting defenses have not been tested against targeted deanonymization, leaving their efficacy in such scenarios unverified.

For targeted deanonymization, Zaheri et al. [33] explored noise-based defenses by introducing cache noise through CPU stress tests [6] web browsing activities. However, they found this method to be ineffective when the attacker retrains their model using the same type of noise used by the defense. In addition, they only considered short-lived attacks, which only use a relatively small observation window. We note that their conclusion only applies to the specific type of noise that was used in that work and cannot be generalized.

3 Background

3.1 CPU-Cache side channels

A cache is a high-speed memory bank that bridges the performance gap between the processor’s speed and the slower main memory. Modern processors usually employ a multi-level cache hierarchy, with the L3 or Last-Level Cache (LLC) being shared across all CPU cores. A cache hit leads to quick data retrieval; if the data is not found in the cache, a cache miss occurs and the data is retrieved from the slower RAM memory.

The very nature of cache sharing introduces unintended communication channels, side channels, which can be exploited to leak sensitive information. Side-channel attacks (SCAs) have been remarkably effective in undermining the security of hardware and software implementations in various cryptosystems [18, 30, 11]. Cache-based SCAs are particularly alarming because they exploit minute variations in cache access times to reveal memory access patterns, potentially exposing critical information such as encryption keys [31].

3.2 The Cache Occupancy Attack

The Prime+Probe attack [18] is a prominent cache side-channel technique that exploits contention in specific cache sets to infer sensitive information. In this attack, the adversary first “primes” the cache by loading their own data into targeted cache sets. After allowing the victim process to execute, the attacker “probes” these sets to detect whether their data has been evicted, indicating that the victim accessed memory mapped to the same sets. By repeating this process, the attacker can discern the victim’s memory access patterns, potentially revealing sensitive information such as encryption keys.

The success of this attack heavily relies on high-resolution timers to distinguish between cache hits and misses. Since the timing differences between accessing data in the cache and in main memory are extremely small — typically around 20 nanoseconds — accurate measurements are crucial. To defend against such timing-based attacks, modern browsers have reduced the resolution of the timers they provide, hindering the effectiveness of attacks like Prime+Probe.

To overcome this limitation, Shusterman et al. [27] proposed the **cache occupancy attack**, which assesses contention across the entire Last-Level Cache (LLC) rather than targeting specific cache sets. This technique involves allocating an LLC-sized buffer and measuring the time required to access the entire buffer (*i.e.*, **the buffer access time**). Cache contention caused by victim processes evicting the attacker’s buffer introduces measurable delays when reading this buffer, allowing the attacker to detect victim activity. Unlike Prime+Probe, the cache occupancy attack is robust to reduced timer resolution such as those available in web browsers.

A variant of the cache occupancy attack, known as **sweep counting**, counts the number of times the buffer can be accessed within a fixed time interval, enabling attacks for even coarser-grained timers, e.g. 100ms, such as those available in the highly-secure Tor Browser.

3.3 The Targeted Deanonymization Attack

Targeted deanonymization attacks are an important class of attacks which threaten user anonymity. An attacker who has complete or partial control over a website seeks to learn the identity of specific target users who are browsing that website. The attacker knows a target only through a public identifier, such as an email address or social media handle.

To mount the attack, the attacker first uses a resource-sharing service like YouTube or Dropbox to privately share a resource (e.g., a YouTube video) with

the target, using the target’s public identifier. The attacker then embeds this resource into the website under their control. When a visitor accesses the attacker-controlled website, the attacker observes whether the resource successfully loads. For example, if an embedded YouTube video — shared privately only with the target — loads, it indicates that the visitor is the intended target.

Zaheri et al. [33] demonstrate that this attack can be executed in less than one second and remains effective even when resource-sharing platforms restrict embedding and when browsers block third-party cookies. To determine if the resource was loaded, the attacker uses the cache occupancy attack and measures the time needed to access the attack buffer while the attack page is loaded by the visitor. A long access time indicates the shared resource was loaded, since by doing so the visitor has evicted the attacker’s buffer from the cache. To analyze the collected cache timings and make an accurate determination, the attacker employs a machine learning classifier.

3.4 The Website Fingerprinting Attack

Website fingerprinting attacks seek to compromise user privacy by using observable activity patterns to identify the websites a user visits. The original website fingerprinting attack assumes an on-path adversary, who leverages statistical analysis of encrypted network traffic metadata — such as packet sizes, timing, and direction — to infer the websites being accessed [12].

Shusterman et al. [27] proposed an alternative attack model which leverages cache-based side channels to bypass network-level defenses. This model assumes that the victim runs a web browser on a target machine and simultaneously accesses both an attacker-controlled site and a sensitive site, each loaded in a separate browser tab. The attacker needs the ability to run JavaScript code in the victim’s browser, thus exploiting the cache occupancy attack channel. By observing cache occupancy patterns, the attacker generates a “fingerprint” — a trace of cache activity over time — which reflects the victim’s visited website.

The attack proceeds in two phases: In the *training phase*, the attacker visits a set of target websites and collects fingerprints corresponding to their cache activity. These traces are labeled and used to train a machine learning classifier, which learns to recognize the unique cache signatures associated with different websites. Next, in the *online phase*, when the victim accesses the attacker-controlled website, the malicious JavaScript code collects the fingerprint of the victim’s cache activity. These traces are then fed into the trained classifier, which identifies the sensitive website the victim is visiting based on its unique cache signature.

4 Threat Model

The threat model for targeted deanonymization and website fingerprinting attacks involves two parties: a user/victim and an attacker. The user browses the internet using a web browser, which may or may not employ anonymity networks such as VPNs or Tor. The attacker is remote and cannot eavesdrop on the user’s network traffic. The user visits an attacker-controlled website containing malicious JavaScript code, which profiles the shared Last-Level Cache (LLC) using the cache occupancy channel [27].

For both attacks, the attacker formulates the problem as a supervised learning task consisting of two phases: a training phase involving offline data collection and model training, and an online phase involving classification of the user’s data using the trained model. We assume that the used data for training the model is collected under settings similar to those of the user, including the same operating system, web browser, and LLC size. The attacker can use known techniques to infer this information about the user [24, 26].

For targeted deanonymization, we assume the adversary has some public information about the victim, such as their Twitter handle or email address. We note the adversary does not need to control the resource-sharing service exploited in the attack; they only need to be a registered user of the service. The adversary faces a binary classification problem: distinguishing between traces belonging to the “target” and those from “non-target” users. During the training phase, the attacker collects cache traces labeled as a “target” or “non-target” user. In the online phase, the attacker gathers an unlabeled cache trace from the user and uses the trained model to predict the labels of this trace, classifying the user as a target or a non-target.

For website fingerprinting, the attacker faces a multiclass classification problem. In the training phase, the attacker collects a set of traces corresponding to sensitive websites on a device identical to the victim’s. The attacker then trains a multi-class classification model using this dataset. In the online phase, the attacker collects a single trace while the victim’s browser renders a website, and this trace is classified using the trained model to identify the website.

5 Targeted Deanonymization: Attacks and Defenses

5.1 Approach Overview

In this section, we perform a systematic investigation of noise-based defenses against cache-based targeted deanonymization attacks. The main research question is whether we can devise a noise-based defense that is resilient against strong attacks. We consider progressively stronger attack strategies and devise defense strategies to mitigate them. This allows us to answer the research question in the positive, and develop a noise-based defense that combines the lessons learned.

At the core of our proposed defense are stress programs, referred to as *stressors*. Each stressor performs read/write operations on memory buffers that are as large as the last-level cache (LLC). Our defense runs while the user is browsing the web, and aims to make cache traces indistinguishable between target and non-target users. We develop seven stressors, which are incorporated into the defense strategies. Section 5.2 provides a description of these stressors. In all defense strategies, we use p to denote the number of stressor instances that are running in parallel, where each stressor instance is typically running on a separate CPU core. Having multiple instances introduces non-determinism into cache access patterns, preventing optimizations like prefetching from affecting the CPU’s cache replacement policy. The increased randomness reduces the attacker’s ability to accurately classify the cache traces, lowering its effectiveness.

We use the term *scenario* to denote the capabilities used by the attacker and the defender (*i.e.*, a pairing of an attack strategy with a defense strategy). As we move through scenarios, we consider progressively stronger attack strategies by incorporating more information about the defense mechanism into the attacker’s machine learning pipeline, by extending the attack duration to capture more information about the target, and by assuming a more advanced threat model. Conversely, we progressively enhance the defense strategy by increasing the amount of added noise (e.g., by increasing the stress intensity), by incorporating randomness into the generated noise, and by periodically switching between stressors.

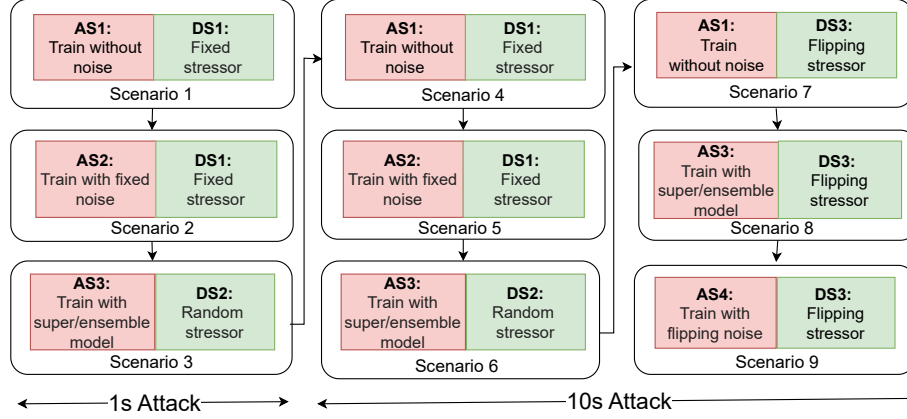


Fig. 1. Flow diagram of the nine scenarios considered, based on four Attack Strategies (AS1-AS4) and three Defense Strategies (DS1-DS3).

Fig. 1 outlines the progression of the nine scenarios considered. Previous work [33] demonstrated that targeted deanonymization attacks could succeed in as little as one second. Thus, we first examine whether noise-based defense strategies can effectively counter such short-duration attacks. The initial “1s Attack” scenarios 1-3 present foundational approaches, starting with basic attacks like training without stress, countered by defenses such as fixed stressors. As the scenarios progress, attack strategies evolve to include methods like training with known fixed stressors or ensemble models, while defense strategies incorporate techniques like randomness to improve resilience. We then evaluate attacks lasting up to 10 seconds in the “10s Attack” scenarios 4-6, which mirror the methods in scenarios 1-3 but allow for an extended cache trace collection. This introduces new challenges for the defenses to remain effective over time.

Finally, scenarios 7-9 introduce our main defense, FLIPSTRESS, which disrupts more sophisticated attack strategies by switching to a randomly-picked stressor at regular intervals.

5.2 Attack and Defense Methodologies

Attack Methodology We follow the attack methodology used by Zaheri *et al.* [33], which we summarize next. The attack begins with the sharing of a leaky resource. The attacker uploads a YouTube (YT) video to the YouTube sharing service and privately shares the video with the target using the victim’s email address. The attacker then embeds the URL of the YT video into a webpage they control. When the target visits this webpage, their browser loads the embedded resource. The attack proceeds in two main phases:

Training Phase: The attacker collects cache activity traces for when the YT video is loaded and when it is not. These traces are used to train a machine learning classifier to identify the cache signature associated with successfully loading the YT video.

Online Phase: The victim visits the attacker-controlled webpage that loads the YT video. As the video is loaded and rendered, the attack script continuously takes cache measurements on the victim’s computer. These measurements are then fed into the trained classifier to determine whether the user is the target. The attack’s effectiveness is determined both by its accuracy and by the *attack duration* parameter, which represents the time for which the attacker collects cache traces. Our cache occupancy code is based on the PP0 repository [21].

The Attack Variant: We consider the basic attack variant that embeds a shared YouTube video using an `<iframe>` in the Chrome browser. Compared to other leaky resources, such as images, we opt for a video since it results in a higher attack accuracy [33].

Later, in Sec. 6, we show that our defense remains effective on a variety of system configurations (*i.e.*, combination of OS, browser, and resource-sharing service).

Defense Methodology: Stressors The defense is implemented as a browser extension that can be activated by the user to run stressor programs during a browsing session. The stressors are implemented as JavaScript web workers. In a browser environment, web workers provide a means to run concurrent scripts in background threads, independent of the main execution thread. Each web worker operates with its own memory space, much like how forked processes do in a native environment. The number of web workers corresponds to the parameter p , representing the degree of parallelism. This parallelism is crucial for generating significant cache contention to disrupt the attacker’s ability to analyze the cache patterns.

We designed and implemented seven stressors to create different types of memory access patterns, each tailored to stress the system in unique ways. For the first three stressors, we turned our attention to the **stress-ng** stress suite [6]. In prior work, Zaheri *et al.* [33] attempted to add artificial noise by using **stress-ng** stressors, but the stressors they considered were found to be ineffective. We re-examined Zaheri *et al.*’s claims regarding **stress-ng** as a source of noise. By broadening the pool of considered stressors, we were able to identify three **stress-ng** stressors which provide resilience. We examined their

source code, and re-implemented them in JavaScript as part of our defense. We developed the other four stressors independently. The seven stressors are: *Read Buffer*, *Write Buffer*, *Read Linked List*, *Write Linked List*, *Stream*, *VM*, and *Memcpy*. These are described in more detail in Appendix B and their source code is provided in the FLIPSTRESS code repository [5].

Experimental Setup We conducted most of the experiments using the Chrome browser, on a Lenovo ThinkPad P14s laptop with an Intel Core i7-1260P CPU running Ubuntu 22.04.1 LTS. To demonstrate the universality of our defense, in Sec. 6 we considered additional system configurations that include the Firefox and Tor browsers, and Windows and MacOS (Sec. 6 and Appendix A).

The attack page contains a YouTube (YT) video that is privately shared with the target. The page also contains a JavaScript script that takes CPU cache measurements. We automated the cache measurements collection using Selenium. We use supervised machine learning to analyze the cache measurement data. To build our data sets, we collect 200 cache occupancy samples while the attack page is loaded — 100 for the target and 100 for the non-target. We used logistic regression for the exploratory portion of the experiments. For each attack setting, we use 90% of the samples for training and 10% for testing. To prevent overfitting the classifier, we used a 10-fold cross validation, and the per-fold accuracies are then combined to produce a single estimate for the mean and standard deviation of the attack accuracy. We also validated the effectiveness of FLIPSTRESS using a convolutional (CNN) and a long short-term memory (LSTM) neural network model. The parameters used for these classifiers are provided in Appendix A. Data analysis was performed using Scikit-Learn v1.5.2 [3] and Tensorflow v2.17.1 [4] with Python v3.10.12 on Google Colab [23].

5.3 Exploring the Space of Attack and Defense Strategies

In this section, we aim to systematically evaluate the effectiveness of different defensive measures against a range of potential attacks. To do so, we explore 9 specific scenarios based on 4 *attack strategies (AS)* and 3 *defense strategies (DS)*, as illustrated in Fig. 1.

Guided by the findings of Zaheri et al. [33], we initially set the attack duration to 1 *second*, which was previously identified as sufficient for mounting a successful attack. An effective defense can theoretically reduce attack accuracy to approximately 50%, equivalent to a random guess by the attacker. Higher accuracy values suggest that the attacker can still extract meaningful information and perform deanonymization to some extent.

Scenario 1: AS1+ DS1 (1s): *AS1 — Train without noise:* The initial attack strategy evaluates an attacker using a model trained on clean cache traces, without defensive noise.

DS1 — Fixed stressor: The defender employs a single, consistent stressor that injects continuous noise into the cache. This strategy aims to disrupt the attacker’s model by masking cache traces with persistent noise, challenging the model’s classification capabilities. The interplay between AS1 and DS1 is quantified in Table 1. We observe that not all stressors demonstrate equal effectiveness.

Table 1. Attack accuracy for Scenario 1 (AS1 + DS1, 1s).

No.	Stressor	$p = 1$	$p = 2$	$p = 4$
1	Read Buffer	99.00%	98.00%	90.50%
2	Write Buffer	50.00%	50.00%	50.00%
3	Read Linked List	50.00%	50.00%	50.00%
4	Write Linked List	50.00%	50.00%	50.00%
5	vm	50.00%	50.00%	50.00%
6	Stream	50.00%	50.00%	50.00%
7	Memcpy	99.50%	82.50%	50.00%
	Average	64.07%	61.50%	55.79%

Table 2. Attack accuracy for Scenario 2 (AS2 + DS1, 1s).

No.	Stressor	$p = 1$	$p = 2$	$p = 4$
1	Read Buffer	95.00%	99.00%	94.50%
2	Write Buffer	91.50%	55.00%	52.50%
3	Read Linked List	99.50%	92.00%	57.50%
4	Write Linked List	86.00%	69.00%	64.50%
5	vm	94.50%	46.00%	52.00%
6	Stream	59.00%	61.50%	64.50%
7	Memcpy	99.50%	99.00%	82.00%
	Average	89.29%	74.50%	66.79%

Specifically, stressors 2 through 6 stand out in their capacity to disrupt the attack and consistently reduce attack accuracy to 50% across all levels of p .

This is because the added noise causes all predictions to pick the target class, despite the fact that only half of the samples are for the target user. The average accuracy over all seven stressors indicates a clear trend: as the number of stressor instances, p , increases, the effectiveness of the defense also improves.

Scenario 2: AS2 + DS1 (1s): In Scenario 2, we introduce a more advanced threat model by assuming that the attacker possesses full knowledge of the defensive measures in place via DS1.

AS2 — Train with fixed noise: The attacker has precise knowledge of the fixed stressor used by the defender. Leveraging this information, the attacker trains a model using cache traces collected in the presence of this specific stressor.

The results in Table 2 show that AS2 proves to be a superior attack strategy compared to AS1. For $p=1$ and $p=2$, the defense is ineffective for most stressors. However, as the value of p increases to 4, there is a notable decline in the attack accuracy. This scenario starkly contrasts with the simpler attack model in AS1, where lower values of p might still offer substantial protection.

Scenario 3: AS3 + DS2 (1s): *DS2 — Random stressor:* The defender deploys a stressor selected at random from the set of available stressors, which operates continuously. The randomness introduces uncertainty for the attacker, as they can no longer predict which specific noise will be present in the cache traces.

AS3 — Train with super/ensemble model: To counter the unpredictability introduced by DS2, the attacker adopts more advanced techniques that aggregate traces from multiple stressors, divided into two approaches - AS3a and AS3b.

Table 3. Attack accuracies for Scenario 3 (AS3 + DS2, 1s).

Attack Strategy	$p = 1$	$p = 2$	$p = 4$
AS3a + DS2	75.29%	69.43%	58.29%
AS3b + DS2	85.71%	77.14%	62.14%

- *AS3a — Super model:* The attacker aggregates cache traces from all stressor types and trains a single “super model”. This model aims to increase predictive accuracy and robustness by pooling data from various stressor-influenced cache traces. This approach allows the attacker to detect patterns across different noise conditions, attempting to classify effectively even under a random environment imposed by DS2.
- *AS3b — Ensemble model:* The attacker builds separate models for each stressor’s cache traces. This strategy leverages the strengths of multiple models, each assessing individual stressor-influenced data independently, which collectively contribute to the final decision based on a majority vote.

The results presented in Table 3 show that both the super model (AS3a) and the ensemble model (AS3b) achieve higher attack accuracies at lower values of p , even without knowledge of the specific stressor used by the defender. However, as the number of stressor instances p increases to 4, we see a noticeable decline in attack accuracy. This indicates that increasing the number of stressor instances remains an effective way to mitigate even these stronger attacks.

5.4 Extending the Attack Duration

So far, we have explored attack and defense strategies within a constrained attack duration of 1 second. Our results show that DS2 is effective against all attack strategies when the number of stressor instances (p) is sufficiently high. However, in real-world scenarios, an attacker could extend the observation period, collecting cache traces over a longer duration, thereby increasing the likelihood of a successful attack.

In this section, we examine the implications of extending the attack duration from 1 second to 10 seconds. We revisit the three scenarios discussed earlier — now referred to as Scenarios 4-6 — with the only difference being the extended attack duration. This allows us to assess how a longer observation window impacts the effectiveness of both the attack and defense strategies.

Scenario 4: AS1 + DS1 (10s): Table 4 indicates that, in this particular scenario, the longer attack duration does not significantly benefit the attacker. Specifically, the consistent attack accuracy of 50% for stressors 2 through 6 remains unchanged compared to the 1-second scenario, implying that these stressors are highly effective against AS1, regardless of the attack duration.

Scenario 5: AS2 + DS1 (10s): The results presented in Table 5 show that extending the attack duration increases the attack accuracy. This suggests that a more extended observation period allows the attacker to gather more data,

Table 4. Attack accuracy for Scenario 4 (AS1 + DS1, 10s).

No.	Stressor	$p = 1$	$p = 2$	$p = 4$
1	Read Buffer	100.00%	100.00%	57.00%
2	Write Buffer	50.00%	50.00%	50.00%
3	Read Linked List	50.00%	50.00%	50.00%
4	Write Linked List	50.00%	50.00%	50.00%
5	vm	50.00%	50.00%	50.00%
6	Stream	50.00%	50.00%	50.00%
7	Memcpy	98.00%	58.00%	50.00%
	Average	64.00%	58.29%	51.00%

Table 5. Attack accuracy for Scenario 5 (AS2 + DS1, 10s).

No.	Stressor	$p = 1$	$p = 2$	$p = 4$
1	Read Buffer	100.00%	100.00%	97.00%
2	Write Buffer	98.00%	84.50%	65.50%
3	Read Linked List	99.50%	99.00%	83.00%
4	Write Linked List	98.00%	96.00%	73.50%
5	vm	100.00%	67.00%	70.00%
6	Stream	63.50%	61.50%	57.00%
7	Memcpy	100.00%	99.00%	85.00%
	Average	94.14%	86.71%	75.86%

thereby effectively countering the fixed stressor defense (DS1). As p increases, the attack accuracy generally decreases, but to a lesser extent Scenario 2. This suggests that while increasing p introduces more noise, the extended time lets the attacker refine their model, mitigating some of the defense’s effectiveness. The results indicate that while DS1 can disrupt shorter attacks, it may not be robust enough for a longer duration.

Scenario 6: AS3 + DS2 (10s): The results presented in Table 6 indicate that in Scenario 6, the attack accuracy consistently improves compared to Scenario 3. This improvement aligns with the trend observed in the preceding scenario, suggesting that the super and ensemble models in AS3 significantly benefit from the extended attack duration.

The findings from Scenarios 4 through 6 imply that both DS1 and DS2 are inadequate in defending against sophisticated attack strategies such as AS3 when the attack duration is extended. This underscores the necessity for a more robust defense mechanism to counter advanced attacks over more extended periods.

Table 6. Attack accuracies for Scenario 6 (AS3 + DS2, 10s).

Attack Strategy	$p = 1$	$p = 2$	$p = 4$
AS3a + DS2	82.29%	77.14%	70.50%
AS3b + DS2	92.85%	93.57%	75.00%

Table 7. Attack accuracy for Scenario 7 (AS1 + DS3, 10s).

$p = 1$	$p = 2$	$p = 4$
50.00%	50.00%	50.00%

Table 8. Attack accuracy for Scenario 8 (AS3a-b + DS3, 10s).

Attack Strategy	$p = 1$	$p = 2$	$p = 4$
AS3a + DS3	64.00%	55.50%	63.50%
AS3b + DS3	50.00%	50.00%	50.00%

5.5 The Flipping Stressor Defense (FLIPSTRESS)

This section introduces our primary defense strategy: the Flipping Stressor Defense, or FLIPSTRESS, which mitigates extended attack durations by dynamically altering the noise patterns throughout the attack window. FLIPSTRESS periodically changes the active stressor, aiming to disrupt the attacker’s ability to collect consistent cache traces over more extended periods, thereby mitigating the increased risk of extended observation times.

Scenario 7: AS1 + DS3 (10s): *DS3 — Flipping stressor:* In this defense strategy, we introduce a more unpredictable countermeasure, in which all stressor instances synchronously switch to a randomly-picked stressor at regular intervals of 0.5 seconds³. This unpredictable change in the stressor environment makes extracting meaningful patterns from the cache traces significantly harder.

Table 7 shows that in Scenario 7, which combines AS1 with DS3, the defense consistently scores 50% accuracy. This suggests that the random and frequent switching of stressors can mitigate the attack, reducing it to a random guess.

Scenario 8: AS3 + DS3 (10s): In this scenario, we evaluate the effectiveness of the flipping stressor defense (DS3) against AS3a (super model) and AS3b (ensemble model) over a 10-second attack duration. Table 8 shows that DS3 is successful against this strong attack, maintaining low accuracy levels.

Scenario 9: AS4 + DS3 (10s): *AS4 — Train with flipping noise:* In this attack strategy, the attacker adjusts their strategy to match the defender’s approach, where the active stressor is flipped randomly at regular intervals. Therefore, the attacker trains the model in the presence of flipping noise.

Table 9 shows that AS4 represents the best attack considered so far against DS3. However, despite the attack’s effectiveness at lower values of p , the DS3 flipping defense remains robust, significantly reducing the attack’s success as p increases to 2 and 4. Given the results across Scenarios 7-9, DS3 consistently demonstrates its strength in mitigating attacks, even against the most effective

³ We selected a 0.5 second switching interval based on experimental observations. In earlier scenarios, we observed that attack accuracies remained relatively low with shorter attack durations, such as 1 second, in Scenarios 1-3. This suggests that brief time frames effectively maintain the defensive advantage – the shorter interval means the attacker has less time to gather consistent data before the stressor changes.

strategies like AS4. The ability of DS3 to maintain low attack accuracies, particularly as stress intensity increases, confirms that it is the most effective defense strategy among those tested.

Table 9. Attack accuracy for scenario 9 (AS4 + DS3, 10s).

$p = 1$	$p = 2$	$p = 4$
79.00%	69.00%	64.00%

6 Results: Targeted Deanonymization

In the previous section, we laid the groundwork for understanding the effectiveness of our defense by focusing exclusively on an attack pipeline that uses Logistic Regression (LR). This initial exploration aimed to demonstrate the iterative process that led to the development of FLIPSTRESS. Building on these insights, this section seeks to rigorously evaluate FLIPSTRESS. We first shift the focus to using more advanced machine learning models. We then assess the defense’s resilience against prolonged attacks, compare its effectiveness with existing defenses, and demonstrate its universality across different system configurations.

Advanced Machine Learning Models. We test the effectiveness of FLIPSTRESS against deep learning models, specifically Convolutional Neural Networks (CNN) [15] and Long Short-Term Memory networks (LSTM) [13]. These models, known for capturing complex patterns in data, present a more rigorous evaluation of our defense under adversarial conditions.

Table 10 shows the attack accuracy when using different machine learning models, as the number of stressor instances, p , increases. Appendix A details the hyperparameters used for these models. CNN and LSTM outperform LR across all p values. While $p=4$ was a strong starting point against LR, achieving meaningful reductions in the performance of advanced models like CNN and LSTM required increasing p up to 12 instances. Appendix C provides additional insights about the impact of FLIPSTRESS on the attacker’s buffer access times.

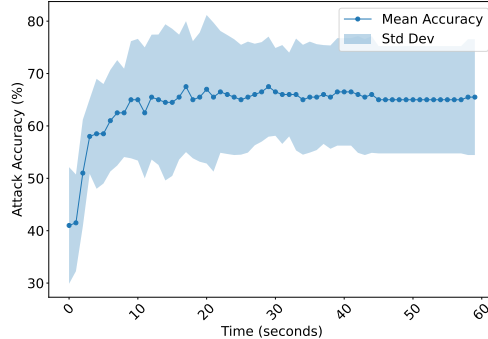
Resilience Against Prolonged Attacks. In practical attack scenarios, the attacker is not constrained to short durations such as 10 seconds. To evaluate the sustainability of FLIPSTRESS, we tested its performance over a prolonged attack duration of 60 seconds. Table 11 shows that FLIPSTRESS continues to disrupt adversarial classification even during extended attacks effectively. Figure 2 shows that the attack accuracy remains relatively constant after an initial rise and some fluctuations. This empirically reinforces FLIPSTRESS as a reliable long-term defense mechanism.

Table 10. Attack accuracy for FLIPSTRESS (p is the number of stressor instances).

p	LR	CNN	LSTM
1	79.0 $\pm$ 10.91%	99.0 $\pm$ 2.0%	88.0 $\pm$ 7.48%
2	69.0 $\pm$ 4.90%	94.0 $\pm$ 5.39%	86.5 $\pm$ 7.43%
4	64.0 $\pm$ 8.60%	81.5 $\pm$ 8.08%	79.5 $\pm$ 9.07%
6	50.5 $\pm$ 10.83%	82.0 $\pm$ 8.72%	89.0 $\pm$ 7.00%
8	52.5 $\pm$ 11.67%	74.5 $\pm$ 10.59%	76.0 $\pm$ 11.58%
10	57.0 $\pm$ 7.14%	57.5 $\pm$ 11.46%	62.5 $\pm$ 8.14%
12	49.0 $\pm$ 7.68%	56.0 $\pm$ 10.68%	57.5 $\pm$ 11.01%

Table 11. FLIPSTRESS attack accuracy for attacks with a duration of 60 seconds.

p	LR	CNN	LSTM
12	65.5% $\pm$ 11.06%	57.5% $\pm$ 7.83%	63.5% $\pm$ 9.23%

**Fig. 2.** FLIPSTRESS attack accuracy over time (using LR).

Comparison with existing defenses. Most prior research into noise-based countermeasures against cache-based attacks on the web has been done in the context of website fingerprinting (WF). However, the underlying principle of these defenses, introducing noise to disrupt the attacker’s ability to extract meaningful patterns in cache timings, can be extended to targeted deanonymization.

Table 12 compares FLIPSTRESS with two existing defenses, adjusted for the targeted deanonymization attack. As the defender does not know the attacker’s website, we use the practical variant of DefWeb [25] that generates noise with random templates across all 12 physical cores, similar to their open-world evaluation. We adjust CacheShaping [16] to match our system configurations and modify its code to utilize all 12 physical cores in our machine, performing read/write operations on 128 files. FLIPSTRESS outperforms these defenses.

Defense Universality Across Diverse System Configurations. To assess the universality of FLIPSTRESS, we tested it across different operating systems (Linux, Windows, MacOS) and browsers (Chrome, Firefox, Tor) under the setups described in Appendix A. Notably, this includes both Intel x86 (Linux, Windows) and ARM (MacOS) which feature distinct cache designs, replacement policies and cache architectures. Table 13 shows that our proposed defense FLIPSTRESS weakens the attack across these diverse system configurations.

7 Results: Website Fingerprinting

To evaluate the effectiveness of FLIPSTRESS against the website fingerprinting attack, we conduct a closed-world evaluation, a widely accepted methodology for

Table 12. Targeted Deanonymization attack accuracy against various defenses (FLIPSTRESS, DefWeb, CacheShaping), under various models (LR, CNN, LSTM) (number of stressor instances $p=12$).

Defense	LR	CNN	LSTM
DefWeb [25]	85.5% $\pm$ 6.5%	95% $\pm$ 4.15%	95% $\pm$ 4.15%
Cache Shaping [16]	62.5% $\pm$ 6.8%	99% $\pm$ 1.5%	100% $\pm$ 0%
FLIPSTRESS	49.0 $\pm$ 7.68%	56.0 $\pm$ 10.68%	57.5 $\pm$ 11.01%

Table 13. FLIPSTRESS Attack Accuracy Across Different System Configurations (system details provided in Appendix A).

OS	Browser	Resource	LR(%)	CNN(%)	LSTM(%)
Linux	Chrome	YouTube video	49 ± 7.68	56 ± 10.68	57.5 ± 11.01
	Chrome	Google Drive video	49 ± 5.25	55 ± 10.50	68.5 ± 8.90
	Firefox	YouTube video	45 ± 5.55	54 ± 9.81	65 ± 9.81
	Firefox	LinkedIn video	51.5 ± 5.50	65 ± 12.00	68 ± 11.00
	Tor	YouTube video	52 ± 5.69	51 ± 11.42	55.93 ± 7.80
Windows	Chrome	YouTube video	48 ± 7.88	56 ± 5.11	73 ± 7.76
	Firefox	YouTube video	51 ± 5.45	55.78 ± 5.65	70 ± 8.90
MacOS	Chrome	YouTube video	62 ± 5.57	70.5 ± 11.06	63.5 ± 7.43

Table 14. Comparison of defenses against the Website Fingerprinting attack, using the CNN model (p is the number of cores/stressor instances).

Defense	p (Utilization)	CNN
DefWeb [25]	6 (100%)	28.8%
CacheShaping [16]	8 (100%)	10.9%
FLIPSTRESS	12 (100%)	6.3%

assessing fingerprinting defenses. Similar to Son *et al.* [25], our evaluation uses 100 websites from Alexa’s most visited websites list [8] and collects 100 traces from each website. We use a standard attack duration of 30s for each website, with cache measurements at a sampling period of 2ms.

Table 14 compares FLIPSTRESS with DefWeb [25] and CacheShaping [16] as a defense against website fingerprinting attacks. The results for DefWeb (variant using practical noise generation) and CacheShaping are sourced directly from their respective studies. The second column of the table specifies the number of CPU cores utilized in each study. In particular, the reported accuracies for DefWeb and CacheShaping are based on scenarios in which the systems used 100% of their available physical cores. To ensure a fair comparison, we also evaluate FLIPSTRESS under 100% core utilization, specifically using $p = 12$ instances, corresponding to the maximum number of physical cores available on the test machine.

FLIPSTRESS outperforms DefWeb and CacheShaping by achieving a significantly lower attack accuracy of 6.3%, compared to 28.8% for DefWeb and 10.9% for CacheShaping with Chrome and Linux.

8 Performance Overhead

As FLIPSTRESS is designed to mitigate browser-based side-channel attacks, we now evaluate its performance overhead in the context of web browsing. We adopt the methodology proposed by Son *et al.* [25], which focuses on assessing the impact of the defense on website loading times. More precisely, we activate the defense and monitor the beginning and completion of the website rendering process by recording the difference in timestamps from the *performance.timing.navigationStart* and *performance.timing.loadEventEnd* functions. Whereas Son *et al.* [25] evaluated overhead using 30 websites, we extend this

Table 15. Performance overhead of FLIPSTRESS with a varying number of stressor instances, averaged over 3 runs (p is the number of stressor instances).

p	Average Loading Time (ms)	Overhead Percentage
0	1,848	0%
1	2,425	31%
2	2,435	32%
4	3,351	81%
6	3,537	91%
8	4,393	138%
10	5,015	171%
12	6,035	226%

assessment to 100 websites based on the Alexa most visited website list [8] to obtain a more robust measurement of performance impact, ensuring our results capture a wider range of website behaviors and variations in load times.

As observed from Table 15, the baseline average loading time without the defense is approximately 1,848 ms. The overhead increases with the number of cores utilized, reaching up to 6,035 ms, or 226%, when all physical cores are used. We measured the overhead of two previously proposed defenses when using all 12 physical cores in our system, and obtained 63% for DefWeb [25] and 119% for CacheShaping [16]. For DefWeb, we use a random noise template while rendering each of the 100 websites.

We note that, while imposing a high overhead when all cores are used, FLIPSTRESS can reduce significantly the attack effectiveness. A mitigating factor is that FLIPSTRESS is meant to be enabled only when visiting sensitive or high-risk websites where cache-based attacks are a concern. As such, its performance impact is limited to when it is actively running.

9 Discussion

Targeted deanonymization shares conceptual similarities with website fingerprinting, as both attacks exploit cache-based side channels to classify browsing activity. However, due to its binary classification nature, targeted deanonymization presents a unique challenge for defenses. In website fingerprinting, the attacker typically faces a multi-class problem involving distinguishing among 100 or more websites, where the complexity lies in differentiating among numerous classes. In contrast, targeted deanonymization simplifies the attacker’s objective to distinguishing between two classes — whether the user is the target or not — significantly reducing the complexity of the classification problem. This presents a more significant challenge for defenders because even slight variations in cache patterns can enable successful classification in targeted deanonymization, making it more difficult to thwart. The difficulty of defending against targeted deanonymization is further amplified by the attackers’ flexibility in creating the attack page. In contrast with website fingerprinting, where the attackers aim to identify public websites whose form and content are known to the defender ahead of time, in the targeted deanonymization scenario, attackers can use different leaky resources, dynamically load these resources at varying times, or alter their website’s structure to generate diverse cache traces. This

adaptability allows attackers to bypass deterministic defenses, such as DefWeb or CacheShaping, which rely on predefined templates or predictable patterns to obfuscate cache behavior. Our experiments show that modifying the attack page by increasing the number of leaky resources or delaying the leaky resource load time leads to only a marginal increase in attack accuracy when FLIPSTRESS is active.

The stressors used in FLIPSTRESS are effective because each stressor is designed to obfuscate the LLC by leveraging diverse and distinct memory access patterns. These patterns interact differently with cache replacement policies and hardware prefetchers ensuring that no single consistent cache behavior emerges. The effectiveness of FLIPSTRESS stems from two key factors. First, it generates random and diverse noise by switching randomly and rapidly among seven distinct stressors. This randomness introduces significant variability in cache access patterns, making it challenging for attackers to identify consistent features for classification. Second, FLIPSTRESS generates higher noise levels, as reflected in its average buffer access time of *23.12 ms*, compared to *3.04 ms* for DefWeb and *10.03 ms* for CacheShaping while using the same number of instances or physical cores. This additional noise is necessary to dominate the attacker’s measurements, ensuring that their data is influenced more by the defense mechanisms than the target’s actual cache behavior. Together, these factors prevent accurate classification by machine learning models, reducing the effectiveness of targeted deanonymization and website fingerprinting attacks. Moreover, FLIPSTRESS’s adaptability makes it more scalable across diverse attack scenarios, as it does not rely on specific assumptions about the attacker’s behavior, the websites being visited, or the micro-architectural details of the victim’s system.

As a noise-based defense, FLIPSTRESS still shares the fundamental limitations of other noise-based defenses: it only reduces the signal-to-noise ratio rather than completely eliminating the side-channel leakage. Thus, as our results show, the accuracy of the more advanced attack strategies we evaluated remains higher than the base rate, even though the application of FLIPSTRESS significantly reduces it. FLIPSTRESS also incurs computational overhead, particularly at higher intensity levels (e.g., $p = 12$), which could impact the user experience in resource-constrained environments. As such, FlipStress is recommended for selective activation when users visit high-risk or sensitive websites. Future work could focus on optimizing the balance between noise generation and system overhead.

10 Conclusion

In this paper, we introduced FLIPSTRESS, a novel defense mechanism against CPU cache-based side-channel attacks on the web, specifically targeted deanonymization and website fingerprinting. By introducing noise injection through randomized stressors, FLIPSTRESS significantly reduces attack success rates, even in scenarios involving advanced adversarial strategies and machine learning models. Its implementation as a browser extension ensures ease of deployment across systems and user accessibility while offering tunable system overhead to balance security with performance.

References

1. XSLeaks Summit 2022. <https://tinyurl.com/xsleakssummit2022> (October 2022)
2. XSLeaks Summit 2023. <https://tinyurl.com/xsleakssummit2023> (September 2023)
3. Scikit-learn: Machine learning in python (2024), <https://scikit-learn.org/>
4. Tensorflow: An end-to-end open source machine learning platform (2024), <https://www.tensorflow.org>
5. Flipstress artifact repository (2025), <https://github.com/tapanbasak12/FlipStress>
6. Colin King: stress-ng. <https://wiki.ubuntu.com/Kernel/Reference/stress-ng>
7. Cook, J., Drean, J., Behrens, J., Yan, M.: There’s always a bigger fish: a clarifying analysis of a machine-learning-assisted side-channel attack. In: Proceedings of the 49th Annual International Symposium on Computer Architecture. p. 204–217. ISCA ’22, Association for Computing Machinery, New York, NY, USA (2022)
8. ExpiredDomains.net: Alexa top websites. <https://www.expireddomains.net/alexa-top-websites/>, accessed: 2022-10-10
9. Godfrey, M., Zulkernine, M.: A server-side solution to cache-based side-channel attacks in the cloud. In: 2013 IEEE Sixth International Conference on Cloud Computing. pp. 163–170 (2013)
10. Godfrey, M., Zulkernine, M.: Preventing cache-based side-channel attacks in a cloud environment. IEEE Transactions on Cloud Computing **2**(4), 395–408 (2014)
11. Gruss, D., Maurice, C., Wagner, K., Mangard, S.: Flush+flush: A fast and stealthy cache attack. In: Proceedings of the 13th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment - Volume 9721. p. 279–299. DIMVA 2016, Springer-Verlag, Berlin, Heidelberg (2016)
12. Hintz, A.: Fingerprinting websites using traffic analysis. In: International workshop on privacy enhancing technologies. pp. 171–178. Springer (2002)
13. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Comput. **9**(8), 1735–1780 (Nov 1997)
14. Knittel, L., Mainka, C., Niemietz, M., Noß, D.T., Schwenk, J.: Xsinator.com: From a formal model to the automatic evaluation of cross-site leaks in web browsers. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. p. 1771–1788. Association for Computing Machinery (2021)
15. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. nature **521**(7553), 436–444 (2015)
16. Li, H., Niu, N., Wang, B.: Cache Shaping: An Effective Defense Against Cache-Based Website Fingerprinting. In: CODASPY 2022 - Proceedings of the 12th ACM Conference on Data and Application Security and Privacy (2022)
17. Li, T., Parameswaran, S.: FaSe: Fast Selective Flushing to Mitigate Contention-based Cache Timing Attacks. In: Proceedings - Design Automation Conference (2022)
18. Liu, F., Yarom, Y., Ge, Q., Heiser, G., Lee, R.B.: Last-level cache side-channel attacks are practical. In: Proceedings - IEEE Symposium on Security and Privacy. vol. 2015-July (2015)
19. Lyu, Y., Mishra, P.: A Survey of Side-Channel Attacks on Caches and Countermeasures. Journal of Hardware and Systems Security **2**(1) (2018)
20. Mukhtar, M.A., Mushtaq, M., Bhatti, M.K., Lapotre, V., Gogniat, G.: FLUSH + PREFETCH: A countermeasure against access-driven cache-based side-channel attacks. Journal of Systems Architecture **104** (2020)

21. Oren, Y.: PP0 GitHub Repository. <https://github.com/Yossioren/pp0> (2021)
22. Osvik, D.A., Shamir, A., Tromer, E.: Cache attacks and countermeasures: the case of aes. In: Proceedings of the 2006 The Cryptographers' Track at the RSA Conference on Topics in Cryptology. p. 1–20. CT-RSA'06, Springer-Verlag, Berlin, Heidelberg (2006)
23. Research, G.: Google colabatory (2024), <https://colab.research.google.com/>
24. Schwarz, M., Lackner, F., Gruss, D.: Javascript template attacks: Automatically inferring host information for targeted exploits. In: NDSS. The Internet Society (2019)
25. Seonghun, S., Debopriya Roy, D., Berk, G.: Defweb: Defending user privacy against cache-based website fingerprinting attacks with intelligent noise injection. In: Proceedings of the 39th Annual Computer Security Applications Conference. p. 379–393. ACSAC '23, Association for Computing Machinery, New York, NY, USA (2023)
26. Shusterman, A., Avraham, Z., Croitoru, E., Haskal, Y., Kang, L., Levi, D., Meltser, Y., Mittal, P., Oren, Y., Yarom, Y.: Website fingerprinting through the cache occupancy channel and its real world practicality. *IEEE Trans. Dependable Secur. Comput.* **18**(5), 2042–2060 (2021)
27. Shusterman, A., Kang, L., Haskal, Y., Meltser, Y., Mittal, P., Oren, Y., Yarom, Y.: Robust website fingerprinting through the cache occupancy channel. In: 28th USENIX Security Symposium (USENIX Security 19). pp. 639–656. USENIX Association, Santa Clara, CA (Aug 2019)
28. Staicu, C.A., Pradel, M.: Leaky images: Targeted privacy attacks in the web. In: Proceedings of the 28th USENIX Security Symposium (2019)
29. Sudhodanan, A., Khodayari, S., Caballero, J.: Cross-Origin State Inference (COSI) Attacks: Leaking Web Site States through XS-Leaks. In: Network and Distributed Systems Security (NDSS) Symposium (2020)
30. Yarom, Y., Falkner, K.: FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In: Proceedings of the 23rd USENIX Security Symposium (2014)
31. YongBin Zhou and DengGuo Feng: Side-Channel Attacks: Ten Years After Its Publication and the Impacts on Cryptographic Module Security Testing (2005), <https://eprint.iacr.org/2005/388>
32. Zaheri, M., Curtmola, R.: Leakuidator: Leaky resource attacks and countermeasures. In: Proc. of the 17th EAI International Conference, SecureComm 2021. vol. 399, pp. 143–163. Springer (2021)
33. Zaheri, M., Oren, Y., Curtmola, R.: Targeted Deanonymization via the Cache Side Channel: Attacks and Defenses. In: Proceedings of the 31st USENIX Security Symposium, Security 2022 (2022)
34. Zhang, Y., Reiter, M.K.: Düppel: Retrofitting commodity operating systems to mitigate cache side channels in the cloud. In: Proceedings of the ACM Conference on Computer and Communications Security (2013)

A Additional Experimental Details

System Information. Table 16 provides information on the hardware, browsers, and operating systems used in the experimental evaluation.

Machine Learning Classifier Parameters. The CNN and LSTM neural network model was used with the hyperparameters described in Table 17. The Logistic Regression classifier was used with 1000 max iterations.

Property	Details
Windows/Linux System	
Device Model	Lenovo ThinkPad P14s Gen 3
Processor	12th Gen Intel(R) Core(TM) i7-1260P, 12 Cores
L1 Cache	448 KB L1i, 640 KB L1d
L2 Cache	9 MB
L3 Cache	18 MB
OS	Ubuntu 22.04.1 LTS / Windows 11 Pro
macOS System	
Device Model	Apple M3 Max
Processor	16-cores (12 P-cores, 4 E-cores)
L1 Cache	192 KB L1i, 128 KB L1d (P-cores) 128 KB L1i, 64 KB L1d (E-cores)
L2 Cache	32 MB (P-cores), 4 MB (E-Cores)
OS	macOS Sonoma 14.5
Browsers	
Google Chrome	124.0.6367.91
Firefox	132.0
Tor	14.0.3

Table 17. Hyperparameters for CNN and LSTM models

Hyperparameters	CNN	LSTM
Optimizer	Adam	Adam
Learning rate	0.001	0.001
Batch size	32	32
Activation function	relu	relu
Dropout	0.3	0.7
Pool size	3	4
Epoch	40	40
Kernel	32, 64, 128	32, 256, 256

B Description of Stressor Programs

The seven stressors implemented as part of the FLIPSTRESS defense are:

1. *Read Buffer*: This stressor repeatedly reads values from two large buffers, which are the size of the LLC buffer, filled with random data.
2. *Write Buffer*: Similar to the Read Buffer, this stressor writes incremented values to two large buffers.
3. *Read Linked List*: This stressor uses two large linked lists and traverses them to read data into a global variable.
4. *Write Linked List*: This stressor also uses linked lists but writes incremented values to each node.
5. *Stream*: This stressor implements a simplified version of the STREAM benchmark, performing a series of operations like copying, scaling, and adding values across large arrays, each the size of the LLC.
6. *VM*: This stressor allocates a LLC-sized buffer and performs bitwise operations that systematically clear bits. The continuous manipulation of bits within the memory region places stress on the cache and memory.

7. *Memcpy*: This stressor tests memory copying operations by implementing naive versions of memcpy and memmove. It continuously copies and moves data between large buffers, stressing the cache and memory.

C Additional Results for the FLIPSTRESS Defense Against the TD Attack

To illustrate the effectiveness of FLIPSTRESS in mitigating the TD attack, we examine the impact of FLIPSTRESS on the attacker’s buffer access times. Without noise introduced by FLIPSTRESS, as seen in Figure 3, the buffer access times for the target and non-target scenarios are easily distinguishable. In contrast, the application of FLIPSTRESS, as illustrated in Figure 4, introduces an unpredictable pattern in the buffer access times measured by the attacker. By rapidly switching and altering cache usage, FLIPSTRESS disrupts the attacker’s cache readings, making them highly irregular. As a result, the buffer access times become randomized, with the target exhibiting higher readings at some times and the non-target at other times. This obfuscation effectively conceals the underlying patterns, rendering them much more challenging for machine learning models to classify accurately.

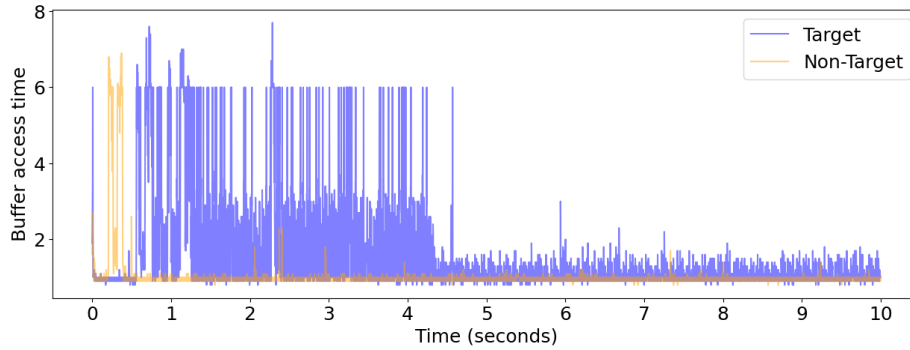


Fig. 3. Buffer access time for target and non-target with no noise.

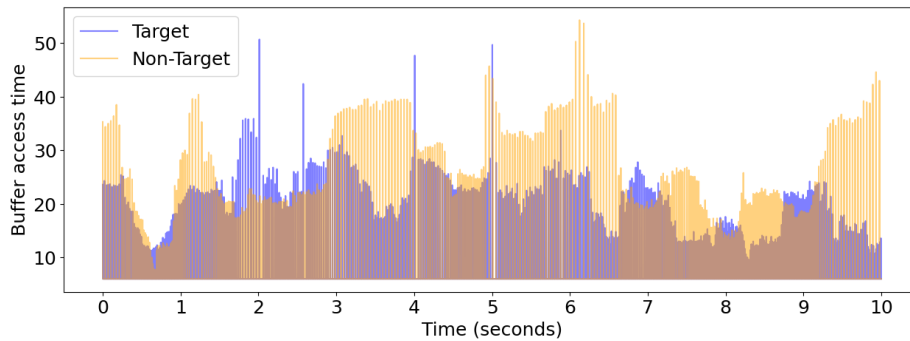


Fig. 4. Buffer access time for target and non-target with FLIPSTRESS ($p=12$).